

ACRME — Epic / Story / Task Backlog

Azure Capacity Reservation Management Engine (ACRME)

Generated 2026-09-07 from the Production-Readiness Review & Final Architecture (PRR).

Single source of truth: [backlog_data.py](#) . This Markdown and the [Jira import CSV](#) are both generated from it, so they never drift.

1. Backlog Summary

Metric	Value
Epics	20
Stories	72
Tasks	193
Total story points	457

Points & stories by delivery phase

Phase	Meaning	Stories	Story points
P1	Pilot — foundation, manual-assist, single/few regions	59	371
P2	Controlled automation — scoring, sharing, quota, multi-region	13	86

2. Legend & Conventions

- **ID scheme** — Epics `ACRME-E##` , Stories `ACRME-S####` , Tasks `ACRME-T#####` .
- **Priority** — Highest / High / Medium / Low (Jira default names).
- **Points** — Fibonacci story points (1, 2, 3, 5, 8, 13).
- **Phase** — P1 Pilot · P2 Controlled automation · P3 Production/Future.
- **PRR refs** — sections of the Production-Readiness Review & Architecture that drive the item.
- **Depends on** — upstream stories that must land first.

3. Epic Index

Epic	Name	Stories	Points
ACRME-E01	Foundation & Platform Infrastructure	5	39
ACRME-E02	Subscription & Onboarding Lifecycle	4	34
ACRME-E03	CRG & Capacity Reservation Management	3	21
ACRME-E04	CRG Sharing & Consumer Authorization	4	21
ACRME-E05	Quota Group Management	4	21
ACRME-E06	Cross-Subscription Zone Alignment	2	13
ACRME-E07	Region Selection & Placement Engine	8	49
ACRME-E08	Placement Scoring & Forecasting	4	23
ACRME-E09	State Model & Concurrency Controls	2	21
ACRME-E10	DR Activation & Fail-back	3	24
ACRME-E11	Tier Escalation (Emergency Capacity)	3	19
ACRME-E12	AKS & VMSS Integration	2	13
ACRME-E13	Data Architecture & Entity Model	2	13
ACRME-E14	API Architecture	3	16
ACRME-E15	Security, RBAC & Managed Identity	3	19
ACRME-E16		3	15

Epic	Name	Stories	Points
	Observability, Dashboards & Alerts		
ACRME-E17	Reconciliation & Scaling	4	23
ACRME-E18	POC & Validation Program	4	29
ACRME-E19	Production Readiness Gates & Governance	3	13
ACRME-E20	v2.4 Reservation-Model Reconciliation	6	31

4. Epics, Stories & Tasks

ACRME-E01 — Foundation & Platform Infrastructure

Goal. Stand up the buildable core: CQRS command/query separation, saga orchestration, the command/event bus, the authoritative engine state store, and provider-isolated Azure adapters.

PRR references. §23 Logical Component Architecture, §24 MG & Subscription Topology, §38 WAF

Rollup. 5 stories · 39 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0101	CQRS Command and Query API skeleton behind API gateway	Highest	8	P1	—
ACRME-S0102	Saga orchestrator and command/event bus	Highest	13	P1	ACRME-S0101
ACRME-S0103	Authoritative engine state store and regional snapshot store	Highest	8	P1	ACRME-S0101
ACRME-S0104	Provider-isolated Azure Resource Manager adapters	High	5	P1	ACRME-S0102
ACRME-S0105	Management-group and subscription topology bootstrap	High	5	P1	—

ACRME-S0101 — CQRS Command and Query API skeleton behind API gateway

As a platform engineer, **I want** a Command API and a Query API fronted by an API gateway, **so that** state-changing and read operations are cleanly separated and independently scalable.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §23, §35
- **Depends on:** —

Acceptance criteria

- Command API and Query API are deployed as separate services behind the API gateway.
- All mutations route only through the Command API; all reads only through the Query API.
- Health/readiness probes return 200 for both services.
- Requests are traced end-to-end with a correlation ID.

Tasks

- [] ACRME-T010101 Scaffold Command API service (routing, DI, config, health probes)
- [] ACRME-T010102 Scaffold Query API service (routing, DI, config, health probes)
- [] ACRME-T010103 Configure API gateway routes, request size limits, and correlation-ID propagation
- [] ACRME-T010104 Add structured logging and distributed tracing baseline

ACRME-S0102 — Saga orchestrator and command/event bus

As a platform engineer, **I want** a persisted saga orchestrator driving operation workers over a command/event bus, **so that** multi-step Azure operations run reliably with checkpoints and compensation.

- **Priority:** Highest · **Points:** 13 · **Phase:** P1
- **PRR refs:** §23, §38 Reliability
- **Depends on:** ACRME-S0101

Acceptance criteria

- Sagas persist each step with a checkpoint and support compensation on failure (closes R-26).
- Operation workers consume commands from the bus and publish events idempotently.
- A partially failed saga can be resumed or compensated without duplicate side effects.
- Saga state is queryable via the operation resource.

Tasks

- [] ACRME-T010201 Implement persisted saga state machine with checkpointing
- [] ACRME-T010202 Implement command/event bus abstraction and worker pool
- [] ACRME-T010203 Implement compensation handlers and partial-saga recovery
- [] ACRME-T010204 Add idempotent event publication and de-duplication

ACRME-S0103 — Authoritative engine state store and regional snapshot store

As a platform engineer, **I want** an authoritative engine-intent store plus a regional snapshot store, **so that** engine intent and cached Azure state are separated with clear ownership.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §23, §34, R-33
- **Depends on:** ACRME-S0101

Acceptance criteria

- Engine state store is authoritative for engine intent; Azure RPs remain authoritative for Azure state.
- Regional snapshot store holds versioned snapshots consumed by placement.
- Throughput is load-tested and autoscale target documented (closes R-33).
- Optimistic concurrency (state version) is enforced on writes.

Tasks

- [] ACRME-T010301 Model and provision the authoritative state store with versioning
- [] ACRME-T010302 Model and provision the versioned regional snapshot store
- [] ACRME-T010303 Load-test throughput and set autoscale targets (R-33)

ACRME-S0104 — Provider-isolated Azure Resource Manager adapters

As a platform engineer, **I want** Azure API adapters isolated per resource provider behind a stable interface, **so that** preview APIs can be flag-gated and swapped without touching business logic.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §23, §21, R-01
- **Depends on:** ACRME-S0102

Acceptance criteria

- Compute, Reservations, Quota, and Resource Graph adapters sit behind stable interfaces.
- Each adapter records API version used per call for evidence.
- Adapters are individually mockable for unit tests.

Tasks

- [] ACRME-T010401 Define adapter interfaces per resource provider
- [] ACRME-T010402 Implement Compute/Reservations/Quota/ARG adapters with API-version capture
- [] ACRME-T010403 Add adapter-level mocks and contract fixtures

ACRME-S0105 — Management-group and subscription topology bootstrap

As a platform operator, **I want** the platform/customer MG and provider/consumer subscription topology provisioned as code, **so that** the engine UAMI and CRGs live in a governed, repeatable hierarchy.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §24
- **Depends on:** —

Acceptance criteria

- Platform MG (provider + engine subs) and customer MG (prod/nonprod/dr subs) provisioned via IaC.
- Engine UAMI created in the engine subscription and referenced by downstream role assignments.
- Topology is parameterised so a customer need not share one hierarchy.

Tasks

- [] ACRME-T010501 Author IaC for management-group and subscription topology
- [] ACRME-T010502 Provision engine User-Assigned Managed Identity
- [] ACRME-T010503 Parameterise topology for per-customer variation

ACRME-E02 — Subscription & Onboarding Lifecycle

Goal. Onboard and offboard customer subscriptions with provider/consumer authorization validation, location-separation enforcement, and full read-back before a customer is marked ready.

PRR references. §22 Must, §26, §43 Onboarding, Runbook B

Rollup. 4 stories · 34 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0201	Subscription onboarding workflow with pre-requisite gates	Highest	13	P1	ACRME-S0102
ACRME-S0202	Provider and consumer authorization validation	Highest	8	P1	ACRME-S0201
ACRME-S0203	Location separation enforcement (hard constraint) at onboarding	Highest	5	P1	ACRME-S0201
ACRME-S0204	Subscription lifecycle monitoring and offboarding	High	8	P1	ACRME-S0202

ACRME-S0201 — Subscription onboarding workflow with prerequisite gates

As a operations engineer, **I want** a gated onboarding workflow that collects inputs and validates every prerequisite, **so that** no customer reaches shared capacity until all checks pass and are recorded.

- **Priority:** Highest · **Points:** 13 · **Phase:** P1
- **PRR refs:** §43 Onboarding, §22 Must
- **Depends on:** ACRME-S0102

Acceptance criteria

- Onboarding collects account details, environments, SKUs/regions, recovery objectives, and consent.
- Workflow blocks progression until zone map, sharing read-back, quota, and DR-floor checks pass.
- Customer marked ready only after all confirmation gates succeed.
- Every gate result is persisted to the audit trail.

Tasks

- [] ACRME-T020101 Model ManagedSubscription entity and onboarding state machine
- [] ACRME-T020102 Implement input-collection and consent capture step
- [] ACRME-T020103 Wire prerequisite gates (zone, sharing, quota, DR floor) as blocking checks
- [] ACRME-T020104 Implement 'ready' marking with full read-back confirmation

ACRME-S0202 — Provider and consumer authorization validation

As a security-conscious operator, **I want** provider registration and consumer deployment rights validated during onboarding, **so that** sharing and deployment cannot proceed on an unauthorized subscription.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §25, §22 Must
- **Depends on:** ACRME-S0201

Acceptance criteria

- Provider subscription and RP registration validated before any sharing action.
- Consumer deployment identity rights on the target CRG validated and read back.
- Validation failure blocks onboarding with a structured precondition error.

Tasks

- [] ACRME-T020201 Validate provider subscription + resource provider registration
- [] ACRME-T020202 Validate consumer deployment identity rights on CRG scope
- [] ACRME-T020203 Emit structured precondition failures on authorization gaps

ACRME-S0203 — Location separation enforcement (hard constraint) at onboarding

As a capacity architect, **I want** onboarding to reject layouts where Prod/NonProd/DR are not correctly separated, **so that** a single regional event cannot take out incompatible environments.

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** §43 Onboarding, HC separation
- **Depends on:** ACRME-S0201

Acceptance criteria

- Prod and DR in the same location is rejected; Prod and NonProd in the same location is rejected.
- NonProd co-located with DR is allowed only with recorded policy + customer acceptance.
- Rejection reason is explicit and audited.

Tasks

- [] ACRME-T020301 Implement separation-rule validator for environment layout
- [] ACRME-T020302 Add NonProd/DR co-location policy exception with acceptance capture

ACRME-S0204 — Subscription lifecycle monitoring and offboarding

As a operations engineer, **I want** lifecycle monitoring plus a controlled offboarding checklist, **so that** suspension/transfer and offboarding never strand running VM restarts.

- **Priority:** High · **Points:** 8 · **Phase:** P1
- **PRR refs:** Runbook B, R-10, R-28
- **Depends on:** ACRME-S0202

Acceptance criteria

- Subscription suspension/transfer triggers revalidation and an alert (closes R-28).
- Offboarding checks active associations before removing sharing (closes R-10).
- Offboarding defaults to deny when active associations exist.

Tasks

- [] ACRME-T020401 Implement subscription lifecycle monitor + revalidation (R-28)
- [] ACRME-T020402 Implement offboarding checklist with association pre-checks (R-10)

ACRME-E03 — CRG & Capacity Reservation Management

Goal. Inventory, create, and safely mutate Capacity Reservation Groups and Capacity Reservations modeled by SKU and zone, with guarded quantity changes and no unsafe auto-decrease in Phase 1.

PRR references. §25, §30, §5 CR quantity update

Rollup. 3 stories · 21 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0301	CRG and CR inventory by SKU and zone	Highest	8	P1	ACRME-S0104
ACRME-S0302	Guarded CR quantity increase	High	8	P1	ACRME-S0301, ACRME-S0902
ACRME-S0303	Guarded CR reduction with zero-reduction and running-VM safety	High	5	P1	ACRME-S0301

ACRME-S0301 — CRG and CR inventory by SKU and zone

As a capacity operator, **I want** authoritative inventory of CRGs and CRs modeled per SKU and zone, **so that** placement and scaling decisions run on accurate capacity state.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §25, §34
- **Depends on:** ACRME-S0104

Acceptance criteria

- CRG hierarchy (Prod/NonProd/DR per region) inventoried with CRs by SKU and zone.
- Provider-side inventory is authoritative; ARG used only for diagnostics (R-03).
- Inventory refresh confirms against ARM before use for mutation (R-04).

Tasks

- [] ACRME-T030101 Model CapacityReservationGroup and CapacityReservation entities
- [] ACRME-T030102 Implement inventory sync with SKU/zone dimensioning
- [] ACRME-T030103 Mark provider inventory authoritative; ARG diagnostics-only (R-03/R-04)

ACRME-S0302 — Guarded CR quantity increase

As a capacity operator, **I want** approval-gated CR quantity increases via tracked operations, **so that** capacity grows safely without cost or capacity surprises.

- **Priority:** High · **Points:** 8 · **Phase:** P1
- **PRR refs:** §30, R-15, G-24
- **Depends on:** ACRME-S0301, ACRME-S0902

Acceptance criteria

- CapacityIncreaseRequest entity with lifecycle, approval, retry, and cancellation (closes G-24).
- Increase requires operator approval and enforces a policy maximum delta (R-15).
- Quantity updated only after validated quota; actual quantity confirmed after change.

Tasks

- [] ACRME-T030201 Model CapacityIncreaseRequest entity + lifecycle (G-24)
- [] ACRME-T030202 Implement approval gate and max-delta guard (R-15)
- [] ACRME-T030203 Implement quota-validated update with post-change confirmation
- [] ACRME-T030204 Add retry and cancellation handling to the request lifecycle

ACRME-S0303 — Guarded CR reduction with zero-reduction and running-VM safety

As a capacity operator, **I want** CR reduction blocked for unknown/destructive scenarios, **so that** reductions never silently remove capacity relied on by running VMs.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §5 CR quantity update, R-09
- **Depends on:** ACRME-S0301

Acceptance criteria

- Reduction below allocated/running usage is blocked with a scenario matrix (R-09).
- Zero-size and unknown-behavior scenarios are blocked pending POC evidence.
- Auto-decrease is disabled in Phase 1.

Tasks

- [] ACRME-T030301 Implement reduction floor validator against allocation/running usage
- [] ACRME-T030302 Encode CR reduction scenario matrix and block unknowns (R-09)

ACRME-E04 — CRG Sharing & Consumer Authorization

Goal. Explicit, read-back-verified CRG sharing across subscriptions with the 100-consumer ceiling handled by sharding, plus safe and forced unsharing.

PRR references. §25 Sharing, §5 CRG sharing, Runbook B, R-02

Rollup. 4 stories · 21 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0401	Explicit CRG sharing workflow with full read-back	Highest	8	P1	ACRME-S0202
ACRME-S0402	100-consumer ceiling monitoring and CRG sharding	High	5	P1	ACRME-S0401
ACRME-S0403	Safe and forced un-sharing with restart-hazard protection	High	5	P1	ACRME-S0401
ACRME-S0404	Sharing drift detection	High	3	P1	ACRME-S0401

ACRME-S0401 — Explicit CRG sharing workflow with full read-back

As a sharing operator, **I want** a sharing sequence that validates, adds, grants, and reads back every step, **so that** a relationship is only 'active' after confirmed state.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §25 Sharing sequence
- **Depends on:** ACRME-S0202

Acceptance criteria

- Sharing adds only explicitly named consumer subscriptions (never tenant-wide).
- Sharing profile and role state are read back and recorded before returning success.
- SharingRelationship entity persisted with active status.

Tasks

- [] ACRME-T040101 Model SharingRelationship entity
- [] ACRME-T040102 Implement validate->add->grant->readback sharing saga
- [] ACRME-T040103 Persist confirmed relationship and expose via Query API

ACRME-S0402 — 100-consumer ceiling monitoring and CRG sharding

As a capacity architect, **I want** continuous relationship-count monitoring and a sharding path, **so that** the pool never silently breaches the 100-consumer platform limit.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** R-02, §5 CRG sharing
- **Depends on:** ACRME-S0401

Acceptance criteria

- Relationship count monitored continuously, not only at create time (R-02).
- Approaching-limit alert fires before the ceiling is hit.
- Documented sharding procedure splits a pool without downtime.

Tasks

- [] ACRME-T040201 Implement consumer-count monitor and threshold alert (R-02)
- [] ACRME-T040202 Document and script CRG sharding procedure

ACRME-S0403 — Safe and forced unsharing with restart-hazard protection

As a sharing operator, **I want** unsharing to default-deny when active associations exist, **so that** removing sharing never strands VM restarts.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** Runbook B, R-10
- **Depends on:** ACRME-S0401

Acceptance criteria

- Forced unsharing rejects by default when active associations exist (Runbook B).
- Consumer owner approval captured before any forced action; impact record retained.
- ForcedUnsharingRequested alert fires on active associations.

Tasks

- [] ACRME-T040301 Implement association enumeration + default-deny gate
- [] ACRME-T040302 Implement approval capture, read-back, and impact record

ACRME-S0404 — Sharing drift detection

As a SRE, **I want** detection when desired and actual sharing profiles differ, **so that** unauthorized or missing consumers are surfaced immediately.

- **Priority:** High · **Points:** 3 · **Phase:** P1
- **PRR refs:** §37 Alerts, R-03
- **Depends on:** ACRME-S0401

Acceptance criteria

- SharingDrift alert on desired/actual mismatch; UnauthorizedConsumer alert on unapproved subscription.
- Drift check runs on the adaptive reconciliation cadence.

Tasks

- [] ACRME-T040401 Implement desired-vs-actual sharing comparator
- [] ACRME-T040402 Wire SharingDrift and UnauthorizedConsumer alerts

ACRME-E05 — Quota Group Management

Goal. Two-group-per-region quota model (Prod group; shared NonProd+DR group) with provider/consumer/group validation and Preview groupType handling.

PRR references. §26, §5 Quota Groups, FC-11, R-05, R-06, R-43

Rollup. 4 stories · 21 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0501	Two-group-per-region quota model	Highest	8	P1	ACRME-S0104
ACRME-S0502	Provider, consumer, and group quota validation	Highest	5	P1	ACRME-S0501
ACRME-S0503	Quota increase endpoint with propagation handling	High	5	P1	ACRME-S0502
ACRME-S0504	Quota Group groupType Preview status handling (FC-11 / R-43)	Medium	3	P1	ACRME-S0501

ACRME-S0501 — Two-group-per-region quota model

As a quota owner, **I want** a Prod quota group and a shared NonProd+DR quota group per region, **so that** budgets are governed separately without DR starving on NonProd demand.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §26
- **Depends on:** ACRME-S0104

Acceptance criteria

- QuotaGroup entities model Prod and shared NonProd+DR groups per region.
- Group formulas and exact controls implemented per §26.
- Membership never treated as proof a specific subscription can deploy (R-06).

Tasks

- [] ACRME-T050101 Model QuotaGroup and SubscriptionQuotaRecord entities
- [] ACRME-T050102 Implement two-group-per-region provisioning + formulas
- [] ACRME-T050103 Enforce mandatory subscription-level checks (R-06)

ACRME-S0502 — Provider, consumer, and group quota validation

As a quota owner, **I want** independent validation of provider, consumer, and group quota, **so that** placement never assumes quota it does not have.

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** §5, R-39
- **Depends on:** ACRME-S0501

Acceptance criteria

- Provider quota not double-counted; API semantics validated (R-39).
- Consumer and group quota validated independently before commit.
- QuotaStateUnknown alert on unavailable/stale reads.

Tasks

- [] ACRME-T050201 Implement provider/consumer/group quota validators
- [] ACRME-T050202 Validate quota API semantics to avoid double count (R-39)
- [] ACRME-T050203 Wire QuotaStateUnknown alert

ACRME-S0503 — Quota increase endpoint with propagation handling

As a quota owner, **I want** a tracked quota-increase submission that polls for confirmed state, **so that** the engine never assumes a propagation SLA.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §30, B-6, R-08
- **Depends on:** ACRME-S0502

Acceptance criteria

- Quota action submitted only when validated as required; polled to confirmed state (B-6).
- Observed propagation distribution recorded; timeout + manual path in runbook.

Tasks

- [] ACRME-T050301 Implement quota-increase submission + async polling
- [] ACRME-T050302 Record propagation distribution; add timeout/manual path (B-6)

ACRME-S0504 — Quota Group groupType Preview status handling (FC-11 / R-43)

As a quota owner, **I want** enforced-vs-advisory groupType treated as a Preview dependency, **so that** the engine never silently relies on preview-only enforcement semantics.

- **Priority:** Medium · **Points:** 3 · **Phase:** P1
- **PRR refs:** FC-11, R-43
- **Depends on:** ACRME-S0501

Acceptance criteria

- groupType enforcement semantics gated behind a Preview feature flag (FC-11).
- POC-30 confirms required API version; governance acceptance recorded if needed (R-43).

Tasks

- [] ACRME-T050401 Flag-gate groupType enforcement reliance (FC-11)

- [] ACRME-T050402 Confirm required API version via POC-30 and record acceptance

ACRME-E06 — Cross-Subscription Zone Alignment

Goal. Build and apply per-subscription physical-to-logical zone mapping so shared reservations land in the correct physical zone; reject on missing mapping.

PRR references. §5 FC-06, §42 POC 6a, R-11, R-41

Rollup. 2 stories · 13 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0601	Zone mapping table built and verified at onboarding	Highest	8	P1	ACRME-S0104
ACRME-S0602	Zone translation applied before every consumer deployment	Highest	5	P1	ACRME-S0601

ACRME-S0601 — Zone mapping table built and verified at onboarding

As a placement owner, **I want** a physical-to-logical zone map per subscription and region built at onboarding, **so that** shared reservations align to the correct physical zone across accounts.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** FC-06, R-41
- **Depends on:** ACRME-S0104

Acceptance criteria

- availabilityZoneMappings read for provider and consumer subscriptions for all managed regions.
- ZoneMappingRecord persisted per subscription/region and read back (FC-06).
- Onboarding blocks if the map cannot be built and verified.

Tasks

- [] ACRME-T060101 Model ZoneMappingRecord entity
- [] ACRME-T060102 Implement Subscriptions-List-Locations zone-mapping fetch
- [] ACRME-T060103 Persist + read-back mapping; block onboarding on failure

ACRME-S0602 — Zone translation applied before every consumer deployment

As a placement engine, **I want** zone translation applied before every zonal deployment, rejecting on missing mapping, **so that** a logical/physical mismatch never causes a silent deployment failure.

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** R-41, POC 6a
- **Depends on:** ACRME-S0601

Acceptance criteria

- Translation resolves the correct consumer logical zone for a provider physical zone.
- Deployment rejected with ZoneMappingUnavailable when mapping is absent (R-41).
- Targeted refresh on churn keeps mapping current (R-11).

Tasks

- [] ACRME-T060201 Implement zone-translation algorithm (physical<->logical)
- [] ACRME-T060202 Enforce ZoneMappingUnavailable rejection path (R-41)
- [] ACRME-T060203 Add event-triggered targeted mapping refresh (R-11)

ACRME-E07 — Region Selection & Placement Engine

Goal. Production-ready region selection: classification model, two-stage eligibility filter, Scenario 1/2 input modes, Middle East cross-geo extension, and the restricted-region exception workflow.

PRR references. §27, §43 Region selection, HC-1..HC-10, VR-1..VR-11

Rollup. 8 stories · 49 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0701	Region classification model (Standard / Restricted / Cross-Geo Extension)	Highest	8	P1	ACRME-S0103
ACRME-S0702	Two-stage eligibility decision tree (HC-1..HC-10)	Highest	8	P1	ACRME-S0701
ACRME-S0703	Scenario 1 — geography-based Prod region derivation	Highest	8	P1	ACRME-S0702, ACRME-S0801
ACRME-S0704	Scenario 2 — specific region input validation	Highest	5	P1	ACRME-S0702
ACRME-S0705	Sequential CVAL then DR selection from Prod anchor	Highest	5	P1	ACRME-S0703, ACRME-S0704
ACRME-S0706	Middle East two-region placement (DR_NOT_OFFERED; Switzerland North cross-geo path pre-configured but inactive)	High	5	P1	ACRME-S0705
ACRME-S0707	Exception-based placement workflow for Restricted re-	High	5	P1	ACRME-S0704

Story	Title	Priority	Points	Phase	Depends on
	gions (EC-1..EC-4)				
ACRME-S0708	Validation Rule Framework (VR-1..VR-11) and gov- ernance con- trols	Medium	5	P1	ACRME-S0701

ACRME-S0701 — Region classification model (Standard / Restricted / Cross-Geo Extension)

As a placement owner, **I want** every managed region classified and Restricted regions pre-filtered before scoring, **so that** constrained regions never enter automated placement.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §27 Classification
- **Depends on:** ACRME-S0103

Acceptance criteria

- Standard, Restricted, and Cross-Geo Extension classes encoded in PlacementPolicy.
- Restricted regions excluded as a pre-filter ahead of all hard constraints (HC-9).
- Unknown region rejected (VR-1); Restricted excluded from scoring (VR-2).

Tasks

- [] ACRME-T070101 Model region classification in PolicyVersion/PlacementPolicy
- [] ACRME-T070102 Implement Stage 1 classification pre-filter (HC-9)
- [] ACRME-T070103 Implement VR-1/VR-2 validation rules

ACRME-S0702 — Two-stage eligibility decision tree (HC-1..HC-10)

As a placement engine, **I want** surviving candidates evaluated against hard constraints HC-1..HC-10 before scoring, **so that** only fully eligible Standard regions reach the scorer.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §27 Decision tree, §27 HC-1..HC-10
- **Depends on:** ACRME-S0701

Acceptance criteria

- Stage 2 applies HC-1..HC-10 (capacity, quota, zone, separation, freshness, geo, DR floor, NonProd/DR integrity, ME cross-geo, extension-path approval).
- Regions failing any HC are excluded from scoring with a recorded reason.
- HC-9 STANDARD_REGION_ONLY and HC-10 CROSS_GEO_EXTENSION_PATH_APPROVED enforced.

Tasks

- [] ACRME-T070201 Implement HC-1..HC-8 constraint checks
- [] ACRME-T070202 Implement HC-9 and HC-10 constraint checks

- [] ACRME-T070203 Record per-region exclusion reasons for audit

ACRME-S0703 — Scenario 1 — geography-based Prod region derivation

As a customer, I want to supply an Azure geography and have the engine derive the Prod anchor, **so that** I get the best Standard region in my geography without naming one.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §27 Prod input modes, §28 PS_Prod, VR-4
- **Depends on:** ACRME-S0702, ACRME-S0801

Acceptance criteria

- Candidate set = Standard regions in geography minus HC exclusions.
- $\text{argmax}(\text{PS_Prod})$; deterministic tie-break by Standard-region list order; cold-start default = first listed.
- Geography-scoped exhaustion error if none eligible (VR-4); never falls outside geography.
- Derived region + all candidate scores + policy version persisted for replay.

Tasks

- [] ACRME-T070301 Implement geography candidate-set builder
- [] ACRME-T070302 Implement $\text{argmax}(\text{PS_Prod})$ selection + deterministic tie-break
- [] ACRME-T070303 Implement geography-scoped exhaustion error (VR-4)
- [] ACRME-T070304 Persist derivation trace to OperationRecord

ACRME-S0704 — Scenario 2 — specific region input validation

As a customer, I want to name a specific region and have it validated or routed to exception, **so that** a Standard region is used directly and a Restricted region needs approval.

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** §27 Prod input modes
- **Depends on:** ACRME-S0702

Acceptance criteria

- Standard region validated against HC-1..HC-10 then set as Prod anchor; PS_Prod used for post-validation only.
- Restricted region routed to the Exception Deployment Workflow.
- Both modes converge on a fixed, validated Prod anchor.

Tasks

- [] ACRME-T070401 Implement Standard-region direct validation path
- [] ACRME-T070402 Route Restricted region input to exception workflow

ACRME-S0705 — Sequential CVAL then DR selection from Prod anchor

As a placement engine, I want CVAL and DR selected sequentially from the fixed Prod anchor, **so that** the three-environment layout respects separation and scoring.

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** §27, §28 PS_NonProd/PS_DR
- **Depends on:** ACRME-S0703, ACRME-S0704

Acceptance criteria

- CVAL selected via PS_NonProd (=PS_CVAL); DR via PS_DR from Standard regions.
- Separation constraints honored across the three environments.
- All three environment scores persisted together for replay.

Tasks

- [] ACRME-T070501 Implement sequential CVAL selection (PS_NonProd)
- [] ACRME-T070502 Implement sequential DR selection (PS_DR)

ACRME-S0706 — Middle East two-region placement (DR_NOT_OFFERED; Switzerland North cross-geo path pre-configured but inactive)

As a placement owner, **I want** Middle East to place Prod + CVAL across its two in-geo regions with DR set to NOT_OFFERED, and the Switzerland North cross-geo path pre-configured but inactive until DEC-001 clears, **so that** Middle East deployments comply with the current DR_NOT_OFFERED legal position (DEC-001) while a compliant DR path is ready to enable by config if approved.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §27 Middle East, HC-10, R-12, DR-014, DEC-001, PLC-010a
- **Depends on:** ACRME-S0705

Acceptance criteria

- Prod = argmax(PS_Prod) over UAE North/Saudi Arabia Central; CVAL = the other in-geo region.
- DR = NOT_OFFERED by default (DR-014, DEC-001): no SourceDestinationDRIndex entry, no DR earmark, no cross-geo activation.
- Switzerland North cross-geo extension is pre-configured but inactive; it activates only via config flip (dr_not_offered['Middle East'] = false) after DEC-001, with no code change, and is then validated against HC-1..HC-10 incl. DR floor.
- The engine never silently substitutes another region; DR_NOT_OFFERED is evaluated before any cross-geo extension.

Tasks

- [] ACRME-T070601 Implement Middle East Prod/CVAL in-geo assignment (two-region)
- [] ACRME-T070602 Implement DR_NOT_OFFERED handling + pre-configured (inactive) Switzerland North cross-geo path gated on DEC-001
- [] ACRME-T070603 Block + alert on degraded extension path when active (no substitution)

ACRME-S0707 — Exception-based placement workflow for Restricted regions (EC-1..EC-4)

As a governance owner, **I want** restricted-region use gated behind four exception conditions, **so that** no Restricted region enters production without explicit approval.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §27 Exception workflow, VR-3
- **Depends on:** ACRME-S0704

Acceptance criteria

- EC-1 explicit request, EC-2 Production-only, EC-3 approval record, EC-4 Scenario-2 input all required (VR-3).

- On pass: assign Exception Prod Anchor, mark Exception Deployment, emit capacity-constraint warning.
- CVAL/DR still chosen from Standard regions; exception approval ID mandatory in OperationRecord.

Tasks

- [] ACRME-T070701 Implement EC-1..EC-4 gate (VR-3)
- [] ACRME-T070702 Implement Exception Deployment marking + warning
- [] ACRME-T070703 Require exception approval ID in OperationRecord commit

ACRME-S0708 — Validation Rule Framework (VR-1..VR-11) and governance controls

As a governance owner, **I want** the full validation-rule framework and PlacementPolicy governance enforced, **so that** region-selection changes require approval and are auditable.

- **Priority:** Medium · **Points:** 5 · **Phase:** P1
- **PRR refs:** §27 VR framework, §27 Governance
- **Depends on:** ACRME-S0701

Acceptance criteria

- VR-1..VR-11 implemented with the specified failure actions.
- Classification/extension changes require PlacementPolicy update + governance approval + Decision Log entry.
- Policy version referenced on every placement decision.

Tasks

- [] ACRME-T070801 Implement remaining VR-5..VR-11 rules
- [] ACRME-T070802 Enforce PlacementPolicy change governance + Decision Log

ACRME-E08 — Placement Scoring & Forecasting

Goal. Implement the corrected, clamped scoring formulas (PS_Prod/PS_NonProd/PS_DR) with demand-weighted distribution, versioned weights, and advisory forecasting.

PRR references. §28, §4 Scoring, R-36, R-37, R-29

Rollup. 4 stories · 23 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0801	Corrected, clamped scoring formulas with versioned weights	Highest	8	P1	ACRME-S0103
ACRME-S0802	Demand-weighted distribution and SKU-dimensional model	High	5	P1	ACRME-S0801
ACRME-S0803	Advisory forecasting with measured accuracy	Medium	5	P2	ACRME-S0801
ACRME-S0804	Shadow joint-optimization comparison	Medium	5	P2	ACRME-S0801

ACRME-S0801 — Corrected, clamped scoring formulas with versioned weights

As a placement owner, **I want** PS_Prod/PS_NonProd/PS_DR with clamped components and versioned weights, **so that** scores are bounded [0,1], replayable, and auditable.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §28 Corrected scoring
- **Depends on:** ACRME-S0103

Acceptance criteria

- Each component clamped via $\text{Clamp}(x) = \max(0, \min(1, x))$; score in [0,1].
- Default weights ($\alpha 0.30, \beta 0.20, \gamma 0.25, \delta 0.15, \epsilon 0.10$) retained for pilot comparison; PS_NonProd uses revised weights.
- Weights carry a PolicyVersion; inputs + version persisted per decision.

Tasks

- [] ACRME-T080101 Implement PS_Prod/PS_NonProd/PS_DR with clamping
- [] ACRME-T080102 Externalise weights into versioned PolicyVersion
- [] ACRME-T080103 Persist score inputs + policy version to OperationRecord

ACRME-S0802 — Demand-weighted distribution and SKU-dimensional model

As a placement owner, **I want** distribution computed from demand units and scoring per SKU/zone, **so that** fairness reflects real demand, not customer count.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §28 Distribution, R-36, R-37
- **Depends on:** ACRME-S0801

Acceptance criteria

- $\text{Distribution} = 1 - \text{Region_Assigned_Demand} / \text{Total_Assigned_Demand}$ (R-37).
- Scoring is SKU-dimensional to avoid simple-formula invalidation (R-36).

Tasks

- [] ACRME-T080201 Implement demand-weighted distribution signal (R-37)
- [] ACRME-T080202 Make scoring SKU/zone-dimensional (R-36)

ACRME-S0803 — Advisory forecasting with measured accuracy

As a capacity planner, **I want** forecast quantity recommendations that stay advisory until accuracy is measured, **so that** forecasts never drive unsafe automatic reduction.

- **Priority:** Medium · **Points:** 5 · **Phase:** P2
- **PRR refs:** §28 Forecast, R-29, ForecastError alert
- **Depends on:** ACRME-S0801

Acceptance criteria

- $\text{Forecast_Quantity} = \text{ceil}(\text{Forecast_Peak} \times (1 + \text{Growth_Buffer}) + \text{DR_Buffer})$ with 30/60/90-day horizons.
- Recommendations advisory-only; reduction never automatic (R-29).
- ForecastError alert when error exceeds policy; accuracy/false-positive tracked.

Tasks

- [] ACRME-T080301 Implement ForecastRecord entity + forecast formula
- [] ACRME-T080302 Keep forecast advisory; wire ForecastError alert (R-29)

ACRME-S0804 — Shadow joint-optimization comparison

As a placement owner, **I want** the sequential method compared against a shadow joint-optimization method, **so that** divergence evidence is gathered before trusting either for auto-commit.

- **Priority:** Medium · **Points:** 5 · **Phase:** P2
- **PRR refs:** §4 Sequential placement, §22 Should
- **Depends on:** ACRME-S0801

Acceptance criteria

- Both methods run without mutating live resources; divergences measured and logged.
- Comparison report available for governance review.

Tasks

- [] ACRME-T080401 Implement shadow joint-optimization evaluator
- [] ACRME-T080402 Log divergences + produce comparison report

ACRME-E09 — State Model & Concurrency Controls

Goal. Implement the engine state machine (EngineModelState) with conditional transitions and dual approval, and atomic capacity holds to prevent over-assignment. Production blocker G-15 / B-7.

PRR references. §29, G-15, B-3, B-7, R-35

Rollup. 2 stories · 21 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S0901	Engine state machine with conditional transitions and dual approval (G-15)	Highest	13	P1	ACRME-S0103
ACRME-S0902	Atomic capacity holds with optimistic concurrency (B-7 / G-20)	Highest	8	P1	ACRME-S0103

ACRME-S0901 — Engine state machine with conditional transitions and dual approval (G-15)

As a SRE, I want EngineModelState with guarded transitions, dual approval, and incident hold, **so that** normal and crisis operations can never illegally mix.

- **Priority:** Highest · **Points:** 13 · **Phase:** P1
- **PRR refs:** §29 State machine, G-15, R-35
- **Depends on:** ACRME-S0103

Acceptance criteria

- States STEADY_STATE, DR_DECLARATION_PENDING, DR_EVENT_ACTIVE, FAILBACK_PENDING, INCIDENT_HOLD implemented.
- Transitions use conditional writes + state version; dual approval required to leave STEADY_STATE.
- Fault-injection test shows no illegal transition (closes G-15/R-35); EngineModeConflict alert wired.
- EngineModelState carries scope, mode, version, incident ID, approvers, lease owner/expiry, recovery checkpoint.

Tasks

- [] ACRME-T090101 Model EngineModelState entity with all required fields
- [] ACRME-T090102 Implement transition guards + conditional writes
- [] ACRME-T090103 Implement dual-approval and incident-hold transitions

- [] ACRME-T090104 Write fault-injection tests proving no illegal transition
- [] ACRME-T090105 Wire EngineModeConflict alert

ACRME-S0902 — Atomic capacity holds with optimistic concurrency (B-7 / G-20)

As a placement engine, **I want** an atomic hold keyed by region/SKU/zone/env/policy before returning an assignment, **so that** concurrent placements cannot double-assign the same capacity.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §29 Capacity holds, B-7, R-23, G-20
- **Depends on:** ACRME-S0103

Acceptance criteria

- PlacementHold created before commit; expires if Azure provisioning does not begin.
- Parallel placement test produces exactly one winner (closes B-7/R-23).
- CustomerRegionAssignment linked to hold IDs (closes G-20); PlacementHoldConflict alert wired.

Tasks

- [] ACRME-T090201 Model PlacementHold + CustomerRegionAssignment entities (G-20)
- [] ACRME-T090202 Implement optimistic-concurrency hold create/expire
- [] ACRME-T090203 Write parallel-placement single-winner test (B-7)
- [] ACRME-T090204 Wire PlacementHoldConflict alert

ACRME-E10 — DR Activation & Failback

Goal. Implement guarded DR declaration, per-workload failover orchestration, engine-enforced DR floor with independent detector, and wave-based failback.

PRR references. §31, Runbook E, §5 DR floor, R-07, R-27, R-38

Rollup. 3 stories · 24 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1001	DR declaration and fail-over orchestration	High	8	P2	ACRME-S0901
ACRME-S1002	Engine-enforced DR floor with independent detector	Highest	8	P1	ACRME-S0901
ACRME-S1003	Wave-based failback with readiness gate	High	8	P2	ACRME-S1001

ACRME-S1001 — DR declaration and failover orchestration

As a DR operator, I want a guarded DR declaration that validates state then orchestrates approved failover, **so that** failover only runs in a validated DR_EVENT_ACTIVE mode.

- **Priority:** High · **Points:** 8 · **Phase:** P2
- **PRR refs:** §31, IncidentRecord G-21
- **Depends on:** ACRME-S0901

Acceptance criteria

- Declaration validates approvals + state version before entering DR_EVENT_ACTIVE.
- Quota, CR, and sharing re-validated from authoritative sources at declaration.
- Per-workload failover status tracked; IncidentRecord persisted (closes G-21).
- Declaration alone does not authorize Tier 2/Tier 3.

Tasks

- [] ACRME-T100101 Model IncidentRecord + DRCapacityPair entities (G-21)
- [] ACRME-T100102 Implement DR declaration validation + mode entry
- [] ACRME-T100103 Implement per-workload failover orchestration

ACRME-S1002 — Engine-enforced DR floor with independent detector

As a SRE, I want an engine-enforced DR floor plus an independent recalculating detector, **so that** NonProd expansion fails closed when the floor is at risk.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §5 DR floor, R-07, R-38, DRFloorViolation alert
- **Depends on:** ACRME-S0901

Acceptance criteria

- Independent detector recomputes the floor from assignments/allocations (R-38).
- Automatic NonProd expansion blocked on detector disagreement (fail-closed, R-07).
- DRFloorViolation alert fires when NonProd use exceeds the effective ceiling.

Tasks

- [] ACRME-T100201 Implement primary DR-floor calculation from potential demand
- [] ACRME-T100202 Implement independent detector + fail-closed block (R-07)
- [] ACRME-T100203 Recompute floor from assignments to avoid staleness (R-38)
- [] ACRME-T100204 Wire DRFloorViolation alert

ACRME-S1003 — Wave-based failback with readiness gate

As a DR operator, I want failback gated on primary readiness and executed in waves, **so that** failback never starts before the primary is truly ready.

- **Priority:** High · **Points:** 8 · **Phase:** P2
- **PRR refs:** Runbook E, R-27
- **Depends on:** ACRME-S1001

Acceptance criteria

- Readiness gate validates app/data/network/DNS/identity/capacity before failback (R-27).
- Restore in waves with health checks; DR traffic drained only after validation.

- DR floor + reservation policy recalculated; return to STEADY_STATE only after all ops close.

Tasks

- [] ACRME-T100301 Implement failback readiness gate + approval (R-27)
- [] ACRME-T100302 Implement wave-based restore + health validation
- [] ACRME-T100303 Implement conservative DR deallocation + floor recalculation

ACRME-E11 — Tier Escalation (Emergency Capacity)

Goal. Implement the tiered emergency-capacity ladder: Tier 1 additive expansion (gated), Tier 2 quota-neutral transfer (disabled until proven), Tier 3 blocked in Phase 1.

PRR references. §32, Runbook C, Runbook D, R-16, R-17, R-18

Rollup. 3 stories · 19 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1101	Tier 1 additive emergency expansion (incident-gated)	High	8	P2	ACRME-S0901, ACRME-S0302
ACRME-S1102	Tier 2 quota-neutral transfer (disabled until proven)	Medium	8	P2	ACRME-S1101, ACRME-S0503
ACRME-S1103	Tier 3 disassociation blocked in Phase 1	Highest	3	P1	ACRME-S0901

ACRME-S1101 — Tier 1 additive emergency expansion (incident-gated)

As a DR operator, **I want** Tier 1 expansion allowed only in DR_EVENT_ACTIVE with fresh checks, **so that** pre-staged headroom is used safely and never outside an incident.

- **Priority:** High · **Points:** 8 · **Phase:** P2
- **PRR refs:** §32, Runbook C, R-16
- **Depends on:** ACRME-S0901, ACRME-S0302

Acceptance criteria

- Tier 1 rejected unless DR_EVENT_ACTIVE (engine-mode guard, R-16).
- Fresh quota/capacity/sharing confirmed; delta bounded by policy maximum.
- CR increase submitted with idempotency key, polled, and read back before VM deployment.

Tasks

- [] ACRME-T110101 Implement Tier 1 engine-mode guard (R-16)
- [] ACRME-T110102 Implement bounded additive expansion + read-back (Runbook C)

ACRME-S1102 — Tier 2 quota-neutral transfer (disabled until proven)

As a DR operator, **I want** Tier 2 reallocation behind a proven+approved flag with impact preview, **so that** NonProd guarantees are never removed unexpectedly.

- **Priority:** Medium · **Points:** 8 · **Phase:** P2
- **PRR refs:** §32, Runbook D, B-2, R-17
- **Depends on:** ACRME-S1101, ACRME-S0503

Acceptance criteria

- Tier 2 disabled until release-and-reuse behavior is measured (B-2) and enabled by flag.
- Requires incident mode + approval + customer-impact preview (R-17).
- DR expanded only after authoritative quota headroom is visible; timeout escalates manually.
- EmergencyCapacityTransfer entity persisted (G-23) with NonProd assurance impact + restoration plan.

Tasks

- [] ACRME-T110201 Model EmergencyCapacityTransfer entity (G-23)
- [] ACRME-T110202 Implement Tier 2 flag + release-and-reuse gate (B-2)
- [] ACRME-T110203 Implement impact preview + approval + manual timeout (Runbook D)

ACRME-S1103 — Tier 3 disassociation blocked in Phase 1

As a security owner, **I want** Tier 3 calls rejected while disabled, **so that** no VM association change happens before the credential model is closed.

- **Priority:** Highest · **Points:** 3 · **Phase:** P1
- **PRR refs:** §32, R-18, Tier3AttemptBlocked alert
- **Depends on:** ACRME-S0901

Acceptance criteria

- Tier 3 request rejected while disabled; Tier3AttemptBlocked alert fires (R-18).
- Feature is disabled-by-default and requires separate board authorization to enable.

Tasks

- [] ACRME-T110301 Implement Tier 3 disabled-by-default rejection
- [] ACRME-T110302 Wire Tier3AttemptBlocked alert (R-18)

ACRME-E12 — AKS & VMSS Integration

Goal. Validate AKS node-pool/autoscaler integration with CRGs and enforce VMSS controls, including rejecting VMSS in emergency transfers (Preview limit).

PRR references. §33, §6 AKS/VMSS, FC-08, R-20, R-21, R-22, R-42

Rollup. 2 stories · 13 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1201	AKS node-pool and autoscaler validation	Medium	8	P2	ACRME-S0502, ACRME-S0602
ACRME-S1202	VMSS controls and emergency-transfer rejection (FC-08)	High	5	P1	ACRME-S1103

ACRME-S1201 — AKS node-pool and autoscaler validation

As a AKS owner, **I want** new node pools validated against zone/SKU/sharing/quota and bounded autoscaler, **so that** AKS never repeatedly requests unavailable nodes or exceeds reservation.

- **Priority:** Medium · **Points:** 8 · **Phase:** P2
- **PRR refs:** §33 AKS, R-21, R-22
- **Depends on:** ACRME-S0502, ACRME-S0602

Acceptance criteria

- New node pool references a CRG only after zone/SKU/sharing/provider+consumer quota checks.
- Autoscaler max bounded by validated reservation + policy over-allocation; retries bounded (R-21).
- Existing node-pool change requires a replacement-impact plan (R-22); node identity separate from ACRME identity.

Tasks

- [] ACRME-T120101 Implement AKS node-pool pre-validation checks
- [] ACRME-T120102 Enforce bounded autoscaler max + bounded retries (R-21)
- [] ACRME-T120103 Require replacement-impact plan for pool updates (R-22)

ACRME-S1202 — VMSS controls and emergency-transfer rejection (FC-08)

As a compute owner, **I want** VMSS operations gated and VMSS excluded from emergency transfers, **so that** a Preview reprovisioning limitation cannot cause a failed DR.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §33 VMSS, FC-08, R-20, R-42, VMSSEmergencyAttempt alert
- **Depends on:** ACRME-S1103

Acceptance criteria

- VMSS included in an emergency transfer is rejected; VMSSEmergencyAttempt alert fires (R-20).
- VMSS reprovisioning via shared CRG during a zone outage documented as Preview-limited (FC-08/R-42).
- Uniform/Flexible create-with-CRG allowed only after mode-specific POC proof.

Tasks

- [] ACRME-T120201 Implement VMSS emergency-transfer rejection + alert (R-20)
- [] ACRME-T120202 Encode VMSS Uniform/Flexible control matrix (Phase 1 gating)

ACRME-E13 — Data Architecture & Entity Model

Goal. Establish the canonical authoritative entity model and close data-model gaps (G-20, G-21, G-23, G-24) with schema, audit, and policy versioning.

PRR references. §34, G-20, G-21, G-23, G-24

Rollup. 2 stories · 13 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1301	Canonical authoritative entity schema and policy versioning	High	8	P1	ACRME-S0103
ACRME-S1302	Append-only audit model with completeness guarantee	High	5	P1	ACRME-S1301

ACRME-S1301 — Canonical authoritative entity schema and policy versioning

As a data owner, **I want** all authoritative entities modeled with a shared schema and PolicyVersion, **so that** engine intent is consistent, versioned, and replayable.

- **Priority:** High · **Points:** 8 · **Phase:** P1
- **PRR refs:** §34 Authoritative entities
- **Depends on:** ACRME-S0103

Acceptance criteria

- All §34 entities have a canonical schema with IDs, timestamps, and version fields.
- PolicyVersion referenced by placement, scoring, and quota decisions.
- Schema migrations are versioned and reversible.

Tasks

- [] ACRME-T130101 Author canonical schema for all authoritative entities
- [] ACRME-T130102 Implement PolicyVersion entity + references
- [] ACRME-T130103 Set up versioned, reversible migrations

ACRME-S1302 — Append-only audit model with completeness guarantee

As a compliance owner, **I want** every accepted mutation to produce an append-only audit record, **so that** incidents and decisions can always be reconstructed.

- **Priority:** High · **Points:** 5 · **Phase:** P1

- **PRR refs:** §34, R-31, §37 Audit completeness SLI
- **Depends on:** ACRME-S1301

Acceptance criteria

- Append-only audit for 100% of accepted mutations (audit completeness SLI, R-31).
- OperationRecord captures before/after state, policy version, and evidence classification.
- Audit is immutable and independently queryable.

Tasks

- [] ACRME-T130201 Implement append-only OperationRecord/audit store
- [] ACRME-T130202 Instrument all mutations to emit audit records (R-31)

ACRME-E14 — API Architecture

Goal. Deliver the canonical async API: operation-resource responses, mandatory mutation metadata, dry-run for high-impact ops, and disabled-by-default emergency endpoints.

PRR references. §35, G-23, R-34

Rollup. 3 stories · 16 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1401	Core async endpoints re-turning operation resources	Highest	8	P1	ACRME-S0101
ACRME-S1402	Mandatory mutation metadata and dry-run	High	5	P1	ACRME-S1401
ACRME-S1403	Emergency endpoints disabled-by-default with contract tests (G-23)	Medium	3	P1	ACRME-S1401

ACRME-S1401 — Core async endpoints returning operation resources

As a API consumer, **I want** state-changing endpoints to return an operation resource, not synchronous completion, **so that** callers poll for true Azure state instead of assuming completion.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §35 Core endpoints
- **Depends on:** ACRME-S0101

Acceptance criteria

- All §35 core endpoints implemented; mutations return an operation + polling URL.

- /placement/select-regions accepts either a specific region (Scenario 2) or a geography (Scenario 1).
- /operations/{id} returns current operation state.

Tasks

- [] ACRME-T140101 Implement core resource endpoints (subscriptions/crgs/quota/...)
- [] ACRME-T140102 Implement /placement/select-regions dual-input contract
- [] ACRME-T140103 Implement /operations/{id} polling resource

ACRME-S1402 — Mandatory mutation metadata and dry-run

As a API consumer, **I want** every mutation to require idempotency key, expected version, policy version, and support dry-run, **so that** high-impact operations are safe, idempotent, and previewable.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §35 Every mutation
- **Depends on:** ACRME-S1401

Acceptance criteria

- Mutations require idempotency key, caller identity, expected state version, policy version, incident ID where applicable.
- Dry-run supported for high-impact operations; structured precondition failures returned.

Tasks

- [] ACRME-T140201 Enforce mandatory mutation metadata middleware
- [] ACRME-T140202 Implement dry-run + structured precondition failures

ACRME-S1403 — Emergency endpoints disabled-by-default with contract tests (G-23)

As a governance owner, **I want** emergency-transfer/increase endpoints in the canonical API but disabled by default, **so that** they exist in the contract and audit model before being enabled.

- **Priority:** Medium · **Points:** 3 · **Phase:** P1
- **PRR refs:** §35, G-23, G-24
- **Depends on:** ACRME-S1401

Acceptance criteria

- /capacity/emergency-transfer and /capacity/increase-requests in the canonical API + authorization matrix.
- Emergency endpoint disabled-by-default via feature flag; contract test present (G-23).

Tasks

- [] ACRME-T140301 Add emergency + increase endpoints to contract + authz matrix
- [] ACRME-T140302 Flag-gate emergency endpoint + add contract test (G-23)

ACRME-E15 — Security, RBAC & Managed Identity

Goal. Implement split UAMIs and application roles with least privilege, and close G-14 (customer-consented, RG-scoped Tier 3 credential model) or keep Tier 3 blocked.

PRR references. §36, §11, G-14, R-19, R-32

Rollup. 3 stories · 19 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1501	Split UAMIs and application roles (least privilege)	Highest	8	P1	ACRME-S0105
ACRME-S1502	G-14 credential model for Tier 3 (customer-consented, RG-scoped)	Highest	8	P2	ACRME-S1501
ACRME-S1503	Break-glass control with time-bound access and post-review	Medium	3	P1	ACRME-S1501

ACRME-S1501 — Split UAMIs and application roles (least privilege)

As a security owner, **I want** the RBAC matrix realized as narrow custom roles and separated identities, **so that** privilege is never concentrated and duties are separated.

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** §36 RBAC matrix, R-19
- **Depends on:** ACRME-S0105

Acceptance criteria

- Reader/Capacity/Sharing/Consumer-Compute/Quota UAMIs + DR/Emergency/Policy-Admin/Auditor roles created per matrix.
- Each role scoped narrowly; prohibited actions verified blocked in a least-privilege test (R-19).
- Role-assignment authority isolated from capacity/VM mutation.

Tasks

- [] ACRME-T150101 Deploy custom role definitions per RBAC matrix
- [] ACRME-T150102 Assign UAMIs at narrowest scope + application roles
- [] ACRME-T150103 Write least-privilege prohibited-action tests (R-19)

ACRME-S1502 — G-14 credential model for Tier 3 (customer-consented, RG-scoped)

As a security owner, **I want** a customer-consented UAMI with RG-scoped custom rights for exact VM association ops, **so that** Tier 3 can eventually be enabled without subscription-wide VM Contributor.

- **Priority:** Highest · **Points:** 8 · **Phase:** P2
- **PRR refs:** §36 MI scope, G-14
- **Depends on:** ACRME-S1501

Acceptance criteria

- Minimum custom action set for VM association defined + RG-scoped; no subscription-wide VM Contributor.
- Customer consent + revocation path implemented and tested in a consumer subscription (closes G-14).
- If the minimum action set cannot be established, Tier 3 stays blocked.

Tasks

- [] ACRME-T150201 Define minimum custom VM-association action set (G-14)
- [] ACRME-T150202 Implement customer consent + revocation flow
- [] ACRME-T150203 Test least-privilege + revocation in consumer subscription

ACRME-S1503 — Break-glass control with time-bound access and post-review

As a security owner, **I want** break-glass access to be time-bound, alerted, and post-reviewed, **so that** emergency bypass cannot be silently abused.

- **Priority:** Medium · **Points:** 3 · **Phase:** P1
- **PRR refs:** §11, R-32
- **Depends on:** ACRME-S1501

Acceptance criteria

- Break-glass role is time-bound and auto-expires; activation raises an alert (R-32).
- Mandatory post-use review recorded in audit.

Tasks

- [] ACRME-T150301 Implement time-bound break-glass role + auto-expiry
- [] ACRME-T150302 Wire activation alert + post-review record (R-32)

ACRME-E16 — Observability, Dashboards & Alerts

Goal. Deliver SLI/SLO instrumentation, operational dashboards, and the full critical alert catalog so operators can see floor integrity, drift, and blocks.

PRR references. §37

Rollup. 3 stories · 15 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1601	SLI/SLO instrumentation	High	5	P1	ACRME-S0101
ACRME-S1602	Operational dashboards	High	5	P1	ACRME-S1601
ACRME-S1603	Critical alert catalog	Highest	5	P1	ACRME-S1601

ACRME-S1601 — SLI/SLO instrumentation

As a SRE, I want the proposed SLIs measured against SLO targets, **so that** engine health is observable and reportable.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §37 SLI/SLO
- **Depends on:** ACRME-S0101

Acceptance criteria

- Query success, mutation availability, placement latency, reconciliation age, and audit completeness measured.
- SLO targets configured with breach reporting; labeled internal (not Microsoft guarantees).

Tasks

- [] ACRME-T160101 Instrument SLI metrics + emit to monitoring
- [] ACRME-T160102 Configure SLO targets + breach reporting

ACRME-S1602 — Operational dashboards

As a operator, I want dashboards for capacity, quota, DR floor, sharing, placement, engine mode, and failures, **so that** state is visible at a glance during steady state and incidents.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §37 Dashboards
- **Depends on:** ACRME-S1601

Acceptance criteria

- Dashboards cover capacity, quota headroom, DR floor integrity, sharing, placement decisions, engine mode, failed ops.
- Dashboards render from live snapshots + engine state.

Tasks

- [] ACRME-T160201 Build capacity/quota/DR-floor dashboards
- [] ACRME-T160202 Build sharing/placement/engine-mode/failed-ops dashboards

ACRME-S1603 — Critical alert catalog

As a SRE, I want the full alert catalog wired with severities and triggers, **so that** critical guardrail violations page the right responders.

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** §37 Alert catalog
- **Depends on:** ACRME-S1601

Acceptance criteria

- All catalog alerts implemented with correct severity + trigger (Engine/DRFloor/Unauthorized/Tier3/VMSS/etc).
- Critical alerts route to on-call; alert tests validate firing conditions.

Tasks

- [] ACRME-T160301 Implement Critical alerts (EngineModeConflict, DRFloorViolation, ...)
- [] ACRME-T160302 Implement High/Medium/Low alerts + routing + tests

ACRME-E17 — Reconciliation & Scaling

Goal. Adaptive, delta-based reconciliation that stays within API budgets at hundreds-thousands of customers, with drift handling and throttle resilience.

PRR references. §10, §7, R-04, R-24, R-25, R-40, FC-16

Rollup. 4 stories · 23 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1701	Adaptive delta-based reconciliation within API budgets	High	8	P1	ACRME-S0103
ACRME-S1702	ARM throttling resilience with per-service budgets	High	5	P1	ACRME-S0104
ACRME-S1703	Drift handling and maintenance mode	Medium	5	P1	ACRME-S1701
ACRME-S1704	Capacity-exhaustion queue and handling	Medium	5	P2	ACRME-S0701

ACRME-S1701 — Adaptive delta-based reconciliation within API budgets

As a SRE, I want targeted, event-triggered reconciliation instead of naive full scans, **so that** reconciliation stays within API budgets at scale.

- **Priority:** High · **Points:** 8 · **Phase:** P1
- **PRR refs:** §10 Adaptive reconciliation, R-24
- **Depends on:** ACRME-S0103

Acceptance criteria

- Reconciliation is delta/event-triggered; full-estate short-interval scan is not used (R-24).
- Critical targeted reconciliation P95 < 2 min; stable-resource age P95 < 15 min.
- ReconciliationStale alert on age beyond policy; ARM confirmation before mutation (R-04).

Tasks

- [] ACRME-T170101 Implement adaptive/delta reconciliation scheduler
- [] ACRME-T170102 Implement event-triggered targeted refresh
- [] ACRME-T170103 Wire ReconciliationStale + ARGDiscoveryLag alerts

ACRME-S1702 — ARM throttling resilience with per-service budgets

As a SRE, I want per-service API budgets, adaptive backoff, and a manual path, **so that** throttling never blocks DR actions.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §7 FC-16, R-25, ARMThrottling alert
- **Depends on:** ACRME-S0104

Acceptance criteria

- Per-service budgets + adaptive backoff seeded from documented throttle baselines (FC-16).
- ARMThrottling alert on retry-budget/throttle-ratio breach; documented manual path independent of engine (R-25).

Tasks

- [] ACRME-T170201 Implement per-service rate budgets + adaptive backoff (FC-16)
- [] ACRME-T170202 Wire ARMThrottling alert + manual fallback runbook (R-25)

ACRME-S1703 — Drift handling and maintenance mode

As a operations engineer, I want maintenance mode and a clear drift policy, **so that** manual Azure changes are not unexpectedly reverted.

- **Priority:** Medium · **Points:** 5 · **Phase:** P1
- **PRR refs:** §10, R-40
- **Depends on:** ACRME-S1701

Acceptance criteria

- Maintenance mode suppresses reconciliation-driven reversal; drift policy documented (R-40).
- Drift surfaced with a clear reconcile-or-accept decision.

Tasks

- [] ACRME-T170301 Implement maintenance mode + drift policy (R-40)

- [] ACRME-T170302 Implement drift surfacing + operator decision path

ACRME-S1704 — Capacity-exhaustion queue and handling

As a operator, **I want** a capacity-exhaustion queue with incident creation and re-evaluation on confirmed state, **so that** exhaustion is handled deterministically, not by elapsed time.

- **Priority:** Medium · **Points:** 5 · **Phase:** P2
- **PRR refs:** Runbook A, §22 Should, CapacityExhausted alert
- **Depends on:** ACRME-S0701

Acceptance criteria

- On no eligible region/CR: freeze holds, rank alternatives without committing, queue + create incident (Runbook A).
- Re-evaluate only after confirmed state change; CapacityExhausted alert fires.

Tasks

- [] ACRME-T170401 Implement capacity-exhaustion queue + incident creation
- [] ACRME-T170402 Implement confirmed-state re-evaluation + alert (Runbook A)

ACRME-E18 — POC & Validation Program

Goal. Execute the critical POC sequence and record structured evidence so preview behaviors and unknowns are proven before production, not asserted.

PRR references. §42, B-1..B-7, all FC items

Rollup. 4 stories · 29 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1801	POC harness with structured evidence capture	High	8	P1	ACRME-S0104
ACRME-S1802	Sharing, quota, and zone-alignment POCs (critical sequence 1-8, incl. 6a)	High	8	P1	ACRME-S1801
ACRME-S1803	DR, concurrency, engine-mode, and tier POCs (critical sequence 9-15)	High	8	P2	ACRME-S1802, ACRME-S0901, ACRME-S0902
ACRME-S1804	VMSS, AKS, and scale POCs (critical sequence 16-18)	Medium	5	P2	ACRME-S1803

ACRME-S1801 — POC harness with structured evidence capture

As a validation engineer, **I want** a POC harness that records API version, region, SKU, zone, IDs, timings, and evidence class, **so that** every result is production-grade evidence, not an assumption.

- **Priority:** High · **Points:** 8 · **Phase:** P1
- **PRR refs:** §42 evidence fields
- **Depends on:** ACRME-S0104

Acceptance criteria

- Harness captures all required evidence fields per run with before/after state + retry history.
- Each result classified as documented/observed/assumed/judgement.

Tasks

- [] ACRME-T180101 Build POC harness + structured evidence store
- [] ACRME-T180102 Implement evidence classification + report export

ACRME-S1802 — Sharing, quota, and zone-alignment POCs (critical sequence 1-8, incl. 6a)

As a validation engineer, **I want** sharing/RBAC, unsharing, quota independence, quantity, and zone-alignment POCs executed, **so that** foundational platform behaviors are proven before pilot.

- **Priority:** High · **Points:** 8 · **Phase:** P1
- **PRR refs:** §42 seq 1-8, B-1, B-2, B-5, FC-06
- **Depends on:** ACRME-S1801

Acceptance criteria

- Sharing/RBAC, unauthorized-consumer rejection, safe+forced unsharing validated.
- Provider/consumer/group quota independence + quantity/reduction/zero-size behavior validated (B-1/B-5).
- Cross-subscription zone alignment (6a) validated with physical-to-logical tables for ≥ 2 subscriptions.

Tasks

- [] ACRME-T180201 Execute POC 1-3 (sharing/RBAC, rejection, unsharing)
- [] ACRME-T180202 Execute POC 4-5,7-8 (quota + quantity, group availability B-1)
- [] ACRME-T180203 Execute POC 6/6a (zone mapping + cross-sub alignment, FC-06)

ACRME-S1803 — DR, concurrency, engine-mode, and tier POCs (critical sequence 9-15)

As a validation engineer, **I want** DR-floor, concurrency, engine-mode, and Tier 1/2/3 POCs executed, **so that** safety-critical behaviors are proven before enabling automation.

- **Priority:** High · **Points:** 8 · **Phase:** P2
- **PRR refs:** §42 seq 9-15, B-7, B-3
- **Depends on:** ACRME-S1802, ACRME-S0901, ACRME-S0902

Acceptance criteria

- DR-floor enforcement, concurrent placement single-winner (B-7), and engine-mode transitions (B-3) validated.
- Tier 1 + Tier 2 validated; Tier 3 confirmed to remain disabled.

Tasks

- [] ACRME-T180301 Execute POC 9-12 (DR floor, quota propagation, concurrency, engine mode)
- [] ACRME-T180302 Execute POC 13-15 (Tier 1, Tier 2, Tier 3-disabled)

ACRME-S1804 — VMSS, AKS, and scale POCs (critical sequence 16-18)

As a validation engineer, **I want** VMSS/AKS behavior and scale testing executed, **so that** workload integration and scale limits are measured, not assumed.

- **Priority:** Medium · **Points:** 5 · **Phase:** P2
- **PRR refs:** §42 seq 16-18, FC-08, R-33
- **Depends on:** ACRME-S1803

Acceptance criteria

- VMSS Uniform/Flexible + AKS node-pool/autoscaler behavior validated (or Preview limits documented).

- Scale testing at target cardinality validates reconciliation + throughput budgets.

Tasks

- [] ACRME-T180401 Execute POC 16-17 (VMSS Uniform/Flexible, AKS)
- [] ACRME-T180402 Execute POC 18 (scale testing) + record budgets

ACRME-E19 — Production Readiness Gates & Governance

Goal. Track gap-closure evidence, preview feature-flag governance, and the production entry gates so pilot→production transitions are evidence-based.

PRR references. §14, §16, §17, §39, §44, R-01, R-34

Rollup. 3 stories · 13 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S1901	Preview feature-flag governance and exit paths	Highest	5	P1	ACRME-S0104
ACRME-S1902	Gap-closure evidence tracker (G-14/G-15/G-20/G-21/G-23/G-24, B-1..B-7)	High	5	P1	—
ACRME-S1903	Production entry gates and conditional pilot checklist	High	3	P1	ACRME-S1902

ACRME-S1901 — Preview feature-flag governance and exit paths

As a product owner, **I want** every preview dependency behind a flag with a governance acceptance + exit path, **so that** preview changes never silently break production.

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** §21, R-01, R-34, R-42, R-43
- **Depends on:** ACRME-S0104

Acceptance criteria

- CRG sharing, groupType enforcement, and VMSS reprovisioning gated behind feature flags (R-01/R-43/R-42).
- Each flag has recorded governance acceptance + a documented exit path.
- Preview/POC results are evidence-labeled and never presented as an SLA (R-34).

Tasks

- [] ACRME-T190101 Implement preview feature-flag framework + registry
- [] ACRME-T190102 Record governance acceptance + exit paths per flag
- [] ACRME-T190103 Enforce evidence-labeling standard (R-34)

ACRME-S1902 — Gap-closure evidence tracker (G-14/G-15/G-20/G-21/G-23/G-24, B-1..B-7)

As a program owner, **I want** a live tracker mapping each gap to its closure control and acceptance evidence, **so that** production gates are provably met, not assumed.

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** §39 Gap closure
- **Depends on:** —

Acceptance criteria

- Every gap/blocker has closure control + acceptance evidence status tracked.
- Tracker blocks phase advancement until required evidence is attached.

Tasks

- [] ACRME-T190201 Build gap-closure tracker mapping gaps->evidence
- [] ACRME-T190202 Gate phase advancement on evidence completeness

ACRME-S1903 — Production entry gates and conditional pilot checklist

As a council/board, **I want** the pilot and production entry gates encoded as an auditable checklist, **so that** pilot success is not mistaken for production approval.

- **Priority:** High · **Points:** 3 · **Phase:** P1
- **PRR refs:** §16, §17, §44 Board actions
- **Depends on:** ACRME-S1902

Acceptance criteria

- Pilot-entry and production-entry gate checklists encoded with named approvers.
- Manual rollback demonstrated; separate production authorization required after pilot.
- Destructive-automation enablement requires separate board authorization.

Tasks

- [] ACRME-T190301 Encode pilot + production entry-gate checklists
- [] ACRME-T190302 Record named approvers + separate-authorization rule

ACRME-E20 — v2.4 Reservation-Model Reconciliation

Goal. Deliver the reservation-model requirements folded into Requirements Baseline v2.4 from the reviewed architecture diagrams: reservation eligibility gates (CAP-020/CAP-021), the seed-at-0 SKU/AZ matrix and product-team budget governance (CAP-022), the explicit regional + per-AZ CRG structure (CAP-023), reactive SKU/AZ discovery (CAP-024), even zone-distribution + rebalancing (PLC-011), and the deterministic naming convention (OPS-006/C-12).

PRR references. Baseline v2.4 §2.4 changelog, CAP-020..024, PLC-011, OPS-006, C-12/C-13, Appendix A.9

Rollup. 6 stories · 31 points

Story	Title	Priority	Points	Phase	Depends on
ACRME-S2001	Reservation eligibility gate — Availability-Set ineligibility & AZ onboarding precondition	Highest	5	P1	ACRME-S0201
ACRME-S2002	Seed-at-0 SKU/AZ matrix and product-team budget governance	Highest	8	P1	ACRME-S0301
ACRME-S2003	Explicit regional + per-AZ CRG structure per environment	High	5	P1	ACRME-S0301
ACRME-S2004	Reactive SKU/AZ discovery auto-create with scope-file governance	High	5	P2	ACRME-S2002, ACRME-S2003
ACRME-S2005	Even zone-distribution target and rebalancing action	High	5	P2	ACRME-S2003, ACRME-S0701
ACRME-S2006	Deterministic RG/CRG/subscription naming convention & counter	Medium	3	P1	—

ACRME-S2001 — Reservation eligibility gate — Availability-Set ineligibility & AZ onboarding precondition

As a onboarding engineer, **I want** the engine to reject reservation onboarding for Availability-Set VMs and require deallocate/redeploy-to-AZ first, **so that** only zone-aligned, reservation-eligible VMs enter the CRG model (CAP-020, CAP-021).

- **Priority:** Highest · **Points:** 5 · **Phase:** P1
- **PRR refs:** CAP-020, CAP-021
- **Depends on:** ACRME-S0201

Acceptance criteria

- VMs deployed in an Availability Set are classified reservation-ineligible and blocked from CRG onboarding (CAP-020).
- Onboarding a running non-zonal VM requires the deallocate-or-redeploy-to-an-availability-zone precondition to be satisfied first (CAP-021).
- The `ReservationEligibility` record captures ineligibility reason and the onboarding precondition state.
- A blocked onboarding returns a deterministic, actionable error and raises a governance item.

Tasks

- [] ACRME-T200101 Implement Availability-Set detection and reservation-ineligibility classification (CAP-020)
- [] ACRME-T200102 Implement deallocate/redeploy-to-AZ onboarding precondition check (CAP-021)
- [] ACRME-T200103 Persist `ReservationEligibility` record with reason + precondition state

ACRME-S2002 — Seed-at-0 SKU/AZ matrix and product-team budget governance

As a capacity governance owner, **I want** an eligible-SKU/AZ seed matrix initialised at count 0 under product-team budget governance, **so that** reservations exist as governed placeholders before demand, extending the CAP-009 seed model (CAP-022).

- **Priority:** Highest · **Points:** 8 · **Phase:** P1
- **PRR refs:** CAP-022, CAP-009
- **Depends on:** ACRME-S0301

Acceptance criteria

- The seed matrix enumerates eligible SKU × AZ combinations with initial reserved count 0 (seed reservations).
- Product-team budget governance approves any increase of a seed reservation above 0.
- `SeedMatrixEntry` records SKU, AZ, environment, governance status, and current count.
- Seed reservations are distinguished from active reservations (`reservationType = SEED`).

Tasks

- [] ACRME-T200201 Model `SeedMatrixEntry` (SKU × AZ × env, count-0 seed) per CAP-022
- [] ACRME-T200202 Implement product-team budget-governance gate for seed increases
- [] ACRME-T200203 Distinguish SEED vs ACTIVE `reservationType` in the reservation model

ACRME-S2003 — Explicit regional + per-AZ CRG structure per environment

As a capacity engineer, **I want** each environment provisioned with one regional CRG plus one CRG per availability zone, **so that** reservations are held at the correct regional/zonal scope, extending CAP-011 (CAP-023).

- **Priority:** High · **Points:** 5 · **Phase:** P1
- **PRR refs:** CAP-023, CAP-011, OPS-006
- **Depends on:** ACRME-S0301

Acceptance criteria

- Each environment (Prod, NonProd, DR) has one regional CRG (`crg-<env>-<region>-reg`) and one CRG per AZ (`az1/az2/az3`).
- `CapacityReservationGroup` carries `environment` and `crgScope` (REGIONAL | ZONAL).
- CRG names are generated deterministically per OPS-006/C-12.
- Per-AZ CRG sizing aligns with the even zone-distribution target (PLC-011).

Tasks

- [] ACRME-T200301 Provision regional + per-AZ CRG set per environment (CAP-023)
- [] ACRME-T200302 Add environment + `crgScope` to CRG model and provisioning
- [] ACRME-T200303 Wire deterministic CRG naming (OPS-006/C-12)

ACRME-S2004 — Reactive SKU/AZ discovery auto-create with scope-file governance

As a capacity engineer, **I want** the engine to auto-create missing SKU/AZ reservations against the seed matrix and raise a scope-file governance item, **so that** newly requested SKU/AZ combinations are captured and governed, reconciling CAP-019 (CAP-024).

- **Priority:** High · **Points:** 5 · **Phase:** P2
- **PRR refs:** CAP-024, CAP-019, CAP-022
- **Depends on:** ACRME-S2002, ACRME-S2003

Acceptance criteria

- A request for a SKU/AZ combination not yet in a CRG triggers auto-creation of the reservation against the seed matrix.
- Each reactive discovery raises a scope-file governance item for review (reconciles CAP-019).
- Auto-created reservations are recorded with provenance (reactive vs. seeded vs. planned).
- Reactive discovery respects reservation eligibility (CAP-020/CAP-021).

Tasks

- [] ACRME-T200401 Implement reactive SKU/AZ discovery + auto-create against seed matrix (CAP-024)
- [] ACRME-T200402 Raise scope-file governance item on reactive discovery (CAP-019 reconciliation)
- [] ACRME-T200403 Record reservation provenance (reactive/seeded/planned)

ACRME-S2005 — Even zone-distribution target and rebalancing action

As a placement engineer, **I want** placement to target an even $\approx 1/\text{zone_count}$ spread and raise a rebalancing action when skew exceeds tolerance, **so that** per-zone failover exposure is bounded and per-AZ CRG sizing stays balanced (PLC-011).

- **Priority:** High · **Points:** 5 · **Phase:** P2
- **PRR refs:** PLC-011, C-13, Appendix A.9, CAP-023
- **Depends on:** ACRME-S2003, ACRME-S0701

Acceptance criteria

- New placement prefers the most under-represented zone (`argmin` VMs per zone) toward $\approx 1/\text{zone_count}$ per zone.
- `Max Skew > Configured Skew Tolerance` (C-13) raises a rebalancing recommendation/action, subject to approval and Azure feasibility.
- The target ratio and tolerance are configuration-driven in `PlacementPolicy`.
- Behaviour matches Appendix A.9 skew formula and is distinct from the ϵ zone-diversity scoring signal (PLC-007).

Tasks

- [] ACRME-T200501 Implement even zone-distribution target + under-represented-zone preference (PLC-011/A.9)
- [] ACRME-T200502 Implement skew-tolerance (C-13) rebalancing recommendation/action
- [] ACRME-T200503 Expose target ratio + tolerance as `PlacementPolicy` config

ACRME-S2006 — Deterministic RG/CRG/subscription naming convention & counter

As a platform operator, **I want** a deterministic naming convention with a counter for resource groups, CRGs, and subscriptions, **so that** resource identities are predictable, collision-free, and auditable (OPS-006, C-12).

- **Priority:** Medium · **Points:** 3 · **Phase:** P1
- **PRR refs:** OPS-006, C-12
- **Depends on:** —

Acceptance criteria

- RG, CRG, and subscription names are generated by a single deterministic convention (OPS-006).
- The naming counter (C-12) guarantees uniqueness and is persisted/auditable.
- Generated names are stable across re-runs for the same inputs.
- RBAC scope assignments consume the deterministic names (no ad-hoc naming).

Tasks

- [] ACRME-T200601 Implement deterministic RG/CRG/subscription naming convention (OPS-006)
- [] ACRME-T200602 Implement + persist the naming counter (C-12)
- [] ACRME-T200603 Align RBAC scope assignment with deterministic names